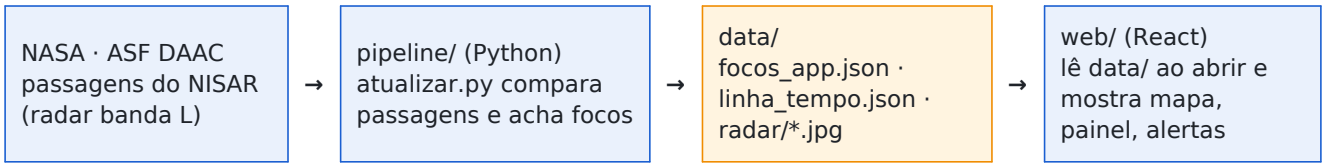
A.R.K.O · Documentação do projeto

Onde fica cada coisa e como modificar · v2.0 · NASA Space Apps 2026 · Dancing with the SARs

Como o projeto funciona

O A.R.K.O tem duas partes que conversam por uma pasta de arquivos. O Python busca e processa os dados reais do satélite NISAR e grava os resultados em `data/`. O app em React lê esses arquivos sempre que abre e mostra mapa, painel, alertas e linha do tempo. Por isso os dados novos aparecem sem precisar recompilar o app.



Duas automações no GitHub Actions mantêm tudo atualizado: Atualizar dados NISAR roda todo dia, procura passagem nova e grava `data/`; Publicar app roda quando o código em `web/` muda, compila o React e grava o resultado em `app/`, que é o que o GitHub Pages mostra.

Pastas do repositório

Pasta / arquivo	O que tem
`web/`	Código do app (React + TypeScript + Tailwind). É aqui que você mexe na interface.
`app/`	App já compilado, publicado no GitHub Pages. Não edite à mão: é gerado a partir de web/.
`pipeline/`	Scripts Python: atualizar.py (automação diária), mapa_base.py (mapas de fundo), buscar.py, processar.py, imagem_app.py e login.py.
`data/`	Resultados dos dados reais: focos_app.json, linha_tempo.json, radar.jpg, radar/AAAA-MM-DD.jpg, focos.geojson, historico.json, passagens.json, estado.json.
`docs/`	Documentos do projeto (este PDF, links do pitch, roadmap e Figma).
`.github/workflows/`	atualizar-nisar.yml (dados, todo dia) e publicar-app.yml (compila o app).
`index.html`	Página raiz que redireciona para app/.

Rodar no seu computador

App (interface). Precisa do Node.js 18 ou mais novo.

```
cd web
npm install        # só na primeira vez
npm run dev        # abre em http://localhost:5173
```

Em modo dev o app lê direto a pasta `data/` do repositório. Para ver a versão final: `npm run build` e depois `npm run preview`.

Pipeline (dados reais). Precisa do Python 3.12 e de uma conta gratuita no NASA Earthdata.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r pipeline/requirements.txt
python pipeline/login.py                # uma vez: salva o login no _netrc
python pipeline/atualizar.py            # só processa se houver passagem nova
python pipeline/atualizar.py --forcar   # reprocessa e refaz as imagens
```

O app (pasta web/)

Arquivo	O que faz
src/App.tsx	Monta a tela: menu lateral, a tela atual, primeiro acesso e a barra do protótipo.
src/lib/store.tsx	Estado global: período, filtros, modo Exemplo × NISAR real, região, idioma, tela atual. Também lê data/*.json ao abrir.
src/lib/regions.ts	Regiões de exemplo (Amazônia e Sul do Brasil): limites do mapa, cidades, focos, alertas, números do painel e comparação.
src/lib/demo.ts	Dados ilustrativos da Amazônia, cores e nomes de cada tipo de evento, períodos (7 dias, 30 dias…).
src/lib/real.ts	Converte os arquivos do pipeline em focos, alertas reais e pontos da linha do tempo.
src/lib/useFocus.ts	Decide o que o mapa mostra: o período inteiro ou só a data escolhida na linha do tempo.
src/lib/i18n.ts	Tradução PT → EN.
src/lib/types.ts	Tipos TypeScript, incluindo o formato dos JSON do pipeline.
src/components/home/	Tela Início: Home, PeriodBar (período e calendário), KpiPanel (painel azul), MapPanel (mapa, focos, legenda, pino), Timeline, AlertPanel, SearchBox (lupa).
src/components/compare/	Tela Comparar: mapa antes/depois e quadro de focos por tipo.
src/components/phone/	Versão celular (abas Alertas, Mapa, Comparar, Sobre).
src/components/Sidebar.tsx	Menu lateral: idioma, Exemplo/NISAR real, região e navegação.
src/components/Sprite.tsx	Todos os ícones: de traço (i-*) e ilustrados 3D (x-fogo, x-agua, x-arvore, x-solo, x-vento, x-tornado, x-alerta).
src/styles/legacy.css	Estilos visuais (vidro, painéis, mapa, marcadores, cards).
tailwind.config.js	Cores e sombras do design para usar em classes Tailwind (ex.: text-fogo, bg-navy-900).
src/assets/	Imagens: logo, mapas base (map-demo.jpg, map-sul.jpg) e radar de reserva.

Onde mudar cada coisa

Quero mudar…	Onde
Um texto da tela	No componente da tela (procure o texto em web/src). Se o texto é novo, adicione a tradução em src/lib/i18n.ts.
A tradução para inglês	src/lib/i18n.ts: textos fixos no objeto EN; textos com números ou datas nas regras RULES.
Cores de um tipo (queimada, enchente…)	src/lib/demo.ts (TYPES) e tailwind.config.js (mesmas cores).
Um ícone	src/components/Sprite.tsx (cada símbolo SVG tem um id).
Painel azul, alertas, mapa (visual)	src/styles/legacy.css (procure .kpis, #alertPanel, .map, .mk, .legend).
Números e focos do modo Exemplo	src/lib/regions.ts (Sul) e src/lib/demo.ts (Amazônia).
Alertas reais (agrupamento e níveis)	src/lib/real.ts, função realAlerts.

Quero mudar…	Onde
A linha do tempo	src/components/home/Timeline.tsx (visual) e src/lib/useFocus.ts (o que aparece no mapa).
A busca da lupa	src/components/home/SearchBox.tsx. Usa o OpenStreetMap (Nominatim) para qualquer lugar do mundo.
Regras de detecção do radar	pipeline/atualizar.py, função comparar (limiares em dB para água, solo e vegetação).
Área real monitorada pelo NISAR	pipeline/atualizar.py: ORBITA, BBOX_BUSCA e IMG (recorte da imagem). Também CIDADES para os nomes dos lugares.
Cores da imagem de radar	pipeline/atualizar.py, função imagem.
Horário da automação diária	.github/workflows/atualizar-nisar.yml (linha cron, em UTC).

Adicionar uma região nova

1. Gere o mapa base no mesmo estilo. Na primeira vez, baixe os dados geográficos para `pipeline/geo/` (instruções no topo do script).

```
python pipeline/mapa_base.py --oeste -80 --leste -66 --lat -10 --estados "" --saida web/src/assets/map-nova.jpg
```

O script imprime os limites (oeste, leste, norte, sul). 2. Em `web/src/lib/regions.ts`, importe a imagem, copie o bloco da região `sul` e ajuste nome, limites, cidades, focos, alertas e números. Para posicionar cidades e focos, use a função `projetar(limites, lon, lat)`, que devolve x e y na grade 746 × 508. 3. Adicione a região em `REGIONS` e `REGION_LIST` no fim do arquivo. Ela aparece sozinha no menu e na busca.

Dados reais em outra área: em `pipeline/atualizar.py`, troque a órbita (ORBITA), o ponto de busca (BBOX_BUSCA) e o recorte (IMG) para a nova área e rode com `--forcar`. O app real passa a mostrar a nova área.

Arquivos de dados (gerados pelo pipeline)

Arquivo	Conteúdo
`focos_app.json`	Comparação principal: datas antes/depois, lista de focos (tipo, lon, lat, área km², grau 0–100, Δ dB, lugar) e limites da imagem. Usado pelo modo NISAR real.
`linha_tempo.json`	Uma entrada por passagem: data, imagem de radar, passagem comparada, focos e resumo. A automação acrescenta as passagens novas.
`radar.jpg`	Radar da passagem mais nova, no estilo do app.
`radar/AAAA-MM-DD.jpg`	Radar de cada passagem da linha do tempo.
`focos.geojson`	Os mesmos focos para abrir no QGIS ou no geojson.io.
`historico.json / passagens.json / estado.json`	Histórico das comparações, passagens encontradas e controle do que já foi processado.

Publicar uma mudança

```
git add -A
git commit -m "descreva a mudança"
git push
```

Se a mudança foi em `web/`, o workflow Publicar app compila e atualiza `app/` sozinho em 1 a 2 minutos. Para rodar a atualização de dados na hora: aba Actions → Atualizar dados NISAR → Run

workflow.

Cuidados

• A senha do NASA Earthdata nunca vai para o código nem para o GitHub: fica no _netrc do computador ou nos Secrets do repositório (EARTHDATA_USERNAME e EARTHDATA_PASSWORD).

• Não envie para o GitHub as pastas web/node_modules, web/dist, pipeline/.cache e pipeline/geo (já estão no .gitignore).

• O modo Exemplo usa dados ilustrativos. Vento e tornado vêm de modelos do tempo (fonte externa), não do NISAR.

• O radar sozinho não separa desmatamento de queimada: por isso o modo real chama de "floresta perdida".

• A busca mundial usa o serviço gratuito do OpenStreetMap: evite automatizar muitas buscas seguidas.